

Code Generation at Scale: Pre-Training and Supervised Fine-Tuning of GPT-2 Style Transformers for Python Code Generation

[Group 5]

Drive:

<https://drive.google.com/drive/folders/1Eu0VHfiG1bAsU0D0nTmwYn2AID0Ety6b>

Git: <https://github.com/Devam0311/NLP-project->

Abstract

This paper presents a systematic scaling study of autoregressive transformer models trained entirely from scratch for Python code generation. We design, implement, and train three GPT-2 style decoder-only models at increasing scales — 50M, 150M, and 225M parameters — on filtered subsets of The Stack Dedup (`bigcode/the-stack-dedup`), training on approximately $\sim 819\text{M}$, $\sim 2\text{B}$, and $\sim 3\text{B}$ tokens of Python code respectively, using the StarCoder2 tokenizer (vocab size: 49,152) with tight sequence packing and no padding. The 50M and 150M models are implemented in pure PyTorch without any pre-built model classes, while the 225M model leverages HuggingFace’s `GPT2LMHeadModel` as its backbone. Pre-training achieved final validation perplexities of 4.28, 3.98, and 3.83 for the three models respectively, demonstrating consistent improvement with scale. All three models were subsequently fine-tuned using Supervised Fine-Tuning (SFT) on the `iamtarun/python_code_instructions_18k_alpaca` dataset with response-only loss masking, with the 225M model achieving a best post-SFT perplexity of 2.12 and 100% syntax pass rate. Our results confirm that scaling from 50M to 225M parameters yields measurable gains in syntactic correctness and instruction-following ability, while also revealing that sub-billion parameter models continue to struggle with complex algorithmic reasoning despite near-perfect surface-level Python syntax.

1 Introduction

Code generation has rapidly emerged as one of the most commercially valuable applications of Large Language Models (LLMs). Systems such as GitHub Copilot, Codex, and GPT-4 have demonstrated that transformer models can produce syntactically valid, contextually relevant code at scale. However, these systems are closed-source and offer no insight into how code generation capability actually emerges during training.

This work takes a fundamentally different approach: rather than fine-tuning an existing pretrained model, we pre-train three models entirely from random initialization, allowing us to directly observe how code understanding develops as a function of both model size and tokens seen. By training a 50M, 150M, and 225M parameter model on the same dataset and tokenizer, we construct a controlled environment for empirical scaling analysis accessible on consumer and cloud GPU hardware. Training from scratch gives us full data control, architectural transparency, and direct empirical insight into how scale affects code quality.

Research Questions

- RQ1.** How does model size affect code quality? Does scaling from 50M to 150M to 225M parameters produce meaningful improvements in syntactic validity and algorithmic correctness?
- RQ2.** What is the minimum viable scale for useful code generation? Can a 50M parameter model trained on $\sim 819\text{M}$ tokens produce syntactically valid and practically useful Python code?
- RQ3.** How transformative is SFT at this scale? Can a small number of instruction-response fine-tuning steps convert a base code model into a coherent interactive assistant?
- RQ4.** Do our results follow Chinchilla scaling predictions?

2 Related Work

GPT-2 (Radford et al., 2019) establishes the decoder-only transformer architecture we adopt across all three models, including Pre-LayerNorm, `gelu_new` activation, weight tying between the token embedding and LM head, and residual projection scaling by $1/\sqrt{2 \times N_{\text{layer}}}$.

StarCoder2 (BigCode, 2024) provides the tokenizer used across all three models — a 49,152-vocabulary BPE tokenizer optimized for source code that minimizes token fragmentation on Python syntax, significantly improving training efficiency over general-purpose tokenizers.

The Stack Dedup (Kocetkov et al., 2022) is our training corpus — a deduplicated, permissively licensed source code dataset where deduplication is critical to prevent models from memorizing boilerplate rather than learning to generalize.

Chinchilla (Hoffmann et al., 2022) establishes that compute-optimal training requires approximately $20\times$ parameters in tokens. Our models are intentionally trained slightly below this threshold, prioritizing inference-time performance over training efficiency.

3 Dataset & Preprocessing

3.1 Data Source

We use The Stack Dedup (`bigcode/the-stack-dedup`), specifically the Python subset (`data/python`). We choose it for three reasons: it is deduplicated at the near-duplicate level preventing memorization of boilerplate, it is permissively licensed making it suitable for research, and its Python subset contains hundreds of gigabytes of real-world code spanning diverse domains including web frameworks, scientific computing, data pipelines, and algorithmic implementations.

3.2 Quality Filtering

Raw code repositories contain a significant proportion of low-quality files. We apply the following heuristic filters ordered cheapest-first for computational efficiency:

Table 1: Quality filter thresholds applied to all models.

Filter	Threshold	Rationale
<code>FILTER_MIN_CHARS</code>	200	Remove empty stubs and trivial files
<code>FILTER_MAX_CHARS</code>	80,000	Exclude files that would crash memory
<code>FILTER_MIN_LINES</code>	8	Require substantive multi-line code blocks
<code>FILTER_MIN_AVG_LINE</code>	10 chars	Filter single-token-per-line scripts
<code>FILTER_MAX_AVG_LINE</code>	200 chars	Filter minified or concatenated files
<code>FILTER_MIN_ALPHA_RATIO</code>	0.20	Remove hex blobs and binary data dumps
Auto-generated markers	Excluded	Files containing “auto-generated”, “do not edit”, etc.
Syntax check	<code>compile()</code>	Remove syntactically invalid Python files

These filters are identical across all three model training runs, ensuring a consistent data distribution for a fair scaling comparison.

3.3 Train / Validation Split

The filtered dataset is split into 99.5% training and 0.5% validation using a fixed seed (`DATA_SEED = 42`) for reproducibility. The validation set is held out entirely and used exclusively for periodic perplexity and syntax pass rate evaluation at each checkpoint.

3.4 Tight Sequence Packing

All sequences are packed into fixed-length blocks of 1,024 tokens with no padding. When a file is shorter than 1,024 tokens, the next file begins immediately in the same block. This achieves 100% compute efficiency — every token in every training batch is a real code token with no wasted padding gradient computations.

3.5 Token Budgets

Table 2: Token budgets across all three models.

Model	Steps	Tokens/Step	Total Tokens
50M	12,500	65,536	~819M
150M	30,517	65,536	~2B
225M	48,828	61,440	~3B

4 Model Architecture

4.1 Architecture Comparison

All three models follow the GPT-2 decoder-only transformer design. The 50M and 150M models are written entirely in raw PyTorch without any HuggingFace model classes, while the 225M model uses HuggingFace’s `GPT2LMHeadModel` as its backbone. All three are trained from random weight initialization — no pretrained weights were used at any point.

Table 3: Architecture comparison across all three models.

Component	50M	150M	225M
Layers	8	12	16
Embedding Dim	512	768	960
Attention Heads	8	12	16
FFN Inner Dim	2,048	3,072	3,840
Parameters	~51M	~161M	~225M
Vocab Size	49,152	49,152	49,152
Context Length	1,024	1,024	1,024
Implementation	Custom PyTorch	Custom PyTorch	HF GPT2LMHeadModel

4.2 Key Architectural Decisions

Pre-LayerNorm. We apply LayerNorm before each sub-layer rather than after. Pre-norm dramatically improves gradient flow in early training — with Post-LayerNorm, gradients at initialization are poorly conditioned and loss spikes are common in the first few thousand steps.

Weight Tying. The language model head shares its weight matrix with the token embedding table: `lm.head.weight = wte.weight`. This saves approximately 25M, 38M, and 47M parameters in the 50M, 150M, and 225M models respectively, and acts as a strong regularizer.

Flash Attention via PyTorch SDPA. All three models use PyTorch 2.0’s `F.scaled_dot_product_attention` with `is_causal=True`, dispatching to Flash Attention kernels depending on hardware. This reduces attention memory complexity from $\mathcal{O}(T^2)$ to near-linear, enabling training with context length 1,024 on 16 GB VRAM without memory overflow.

NewGELU Activation. We use the `gelu_new` tanh approximation matching the original GPT-2 activation:

$$\text{NewGELU}(x) = 0.5 x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715 x^3) \right) \right)$$

with maximum absolute error versus exact GELU of approximately 1.8×10^{-4} per element.

Residual Projection Scaling. Output projections are scaled down by $1/\sqrt{2 \times N_{\text{layer}}}$ at initialization, keeping residual stream variance approximately constant regardless of depth.

5 Training Setup

5.1 Hardware

All models were trained on single GPUs without distributed training. The 50M model was trained on a Kaggle T4 GPU (16 GB VRAM), and the 150M and 225M models were trained on a local NVIDIA RTX 4060 Ada (16 GB VRAM).

5.2 Mixed Precision

Table 4: Mixed precision configuration per model.

Model	Precision	Reason
50M	float16 + GradScaler	T4 does not support BFloat16 natively
150M	bfloat16	Auto-detected; no GradScaler needed, more stable
225M	bfloat16	Native on RTX 4060 Ada

5.3 Optimizer

All three models use AdamW with the following hyperparameters:

Table 5: AdamW optimizer hyperparameters.

Parameter	Value
LR_INIT	3×10^{-4}
LR_MIN	3×10^{-5}
β_1, β_2	0.9, 0.95
weight_decay	0.1
eps	10^{-8} (10^{-6} for 150M from step 25,000 [†])
grad_clip	1.0
fused	True (CUDA fused kernel)

[†]The 150M model’s epsilon was increased from 10^{-8} to 10^{-6} at step 25,000 as part of a stabilization intervention. See Section 6.1.

Weight decay is applied only to 2D+ weight matrices — biases and LayerNorm parameters are excluded, following standard practice.

5.4 Learning Rate Schedule

We implement a linear warmup followed by cosine decay for all models. LR increases linearly from 0 to LR_INIT over a warmup period, then decreases to LR_MIN following a cosine curve.

Table 6: Learning rate schedule per model.

Model	Warmup Steps	Total Steps
50M	700	12,500
150M	2,000	30,517
225M	2,000	48,828

5.5 Batch Configuration & Total Compute

Table 7: Batch configuration and total compute per model.

Model	Micro-batch	Grad Accum	Eff. Batch	Tokens/Step	Steps	Total Tokens
50M	8	8	64 seqs	65,536	12,500	~819M
150M	2	32 [‡]	64 seqs	65,536	30,517	~2B
225M	1	60	60 seqs	61,440	48,828	~3B

[‡]The 150M model used GRAD_ACCUM=16 for steps 0–25,000, then increased to 32 from step 25,000 onwards. Total token count remains ~2B. See Section 6.1.

5.6 Gradient Clipping

All models apply hard gradient clipping at `max_norm` = 1.0 before each optimizer step. This is the primary defense against loss spikes corrupting the optimizer state.

6 Training Analysis

6.1 Training Instability & Resolution (150M Model)

At step 25,900, gradient norm spiked from a stable ~ 1.0 to a peak of 995.6 over just 600 steps:

Table 8: Gradient norm explosion in the 150M model.

Step	Grad Norm
25,800	1.017
25,900	112.166
26,500	995.610

Root Cause. The explosion occurred where learning rate was still relatively high at 1.46×10^{-4} during cosine decay. Outlier batches produced large gradient updates that corrupted Adam’s internal moment estimates despite gradient clipping. A parallel training run without gradient clipping experienced catastrophic explosion reaching `grad_norm` > 3,500, resulting in complete model corruption.

Three-Part Intervention. We rolled back to the step 25,000 checkpoint and applied three targeted fixes:

1. **Doubled gradient accumulation** (GRAD_ACCUM 16→32): Larger effective batch reduces per-step gradient variance.
2. **Increased Adam epsilon** ($10^{-8} \rightarrow 10^{-6}$): Prevents near-zero denominator instability specific to BF16's reduced dynamic range.
3. **Gradient skip block**: Steps where `grad_norm > 500` are skipped entirely, protecting Adam's moment estimates.

Results.

Table 9: Metrics before and after the stabilization intervention.

Metric	Before	After
Grad norm (next step)	995.6	0.549
Val loss	1.4587	1.3823
Perplexity	4.30	3.98

6.2 Checkpoint History

Table 10: 150M model checkpoint history (selected checkpoints).

Step	Tokens Seen	Val Loss	Val PPL	Syntax Pass Rate
10,000	~655M	1.5962	4.93	20%
20,000	~1.31B	1.4714	4.36	20%
25,000	~1.64B	1.4587	4.30	20%
27,500	~1.80B	1.3823	3.98	20%

Table 11: 225M model key checkpoints (selected checkpoints).

Step	Tokens Seen	Val Loss	Val PPL	Syntax Pass Rate
20,000	~1.23B	1.5323	4.63	40%
30,000	~1.84B	1.4243	4.15	40%
40,000	~2.46B	1.3594	3.89	20%
45,000	~2.76B	1.3441	3.83	20%

50M Model: Final Val Loss: **1.4535**, Final Perplexity: **4.28**.

6.3 Loss Curve Analysis

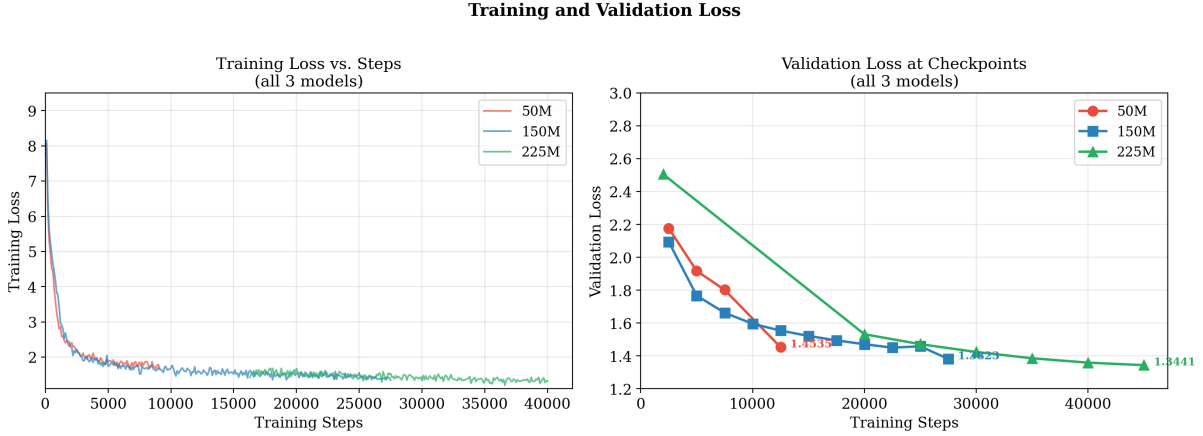


Figure 1: Training loss curves for 50M, 150M, and 225M models. All models show stable convergence despite the mid-training instability in the 150M run.

The validation loss curves for all three models follow a standard power-law descent consistent with neural scaling literature. The 225M model achieves its lowest perplexity of **3.83** at step 45,000, while the 150M achieves **3.98** at step 27,500 — a gap of 0.15 perplexity points consistent with the expected benefit of additional parameters.

6.4 Validation Perplexity vs. Training Steps

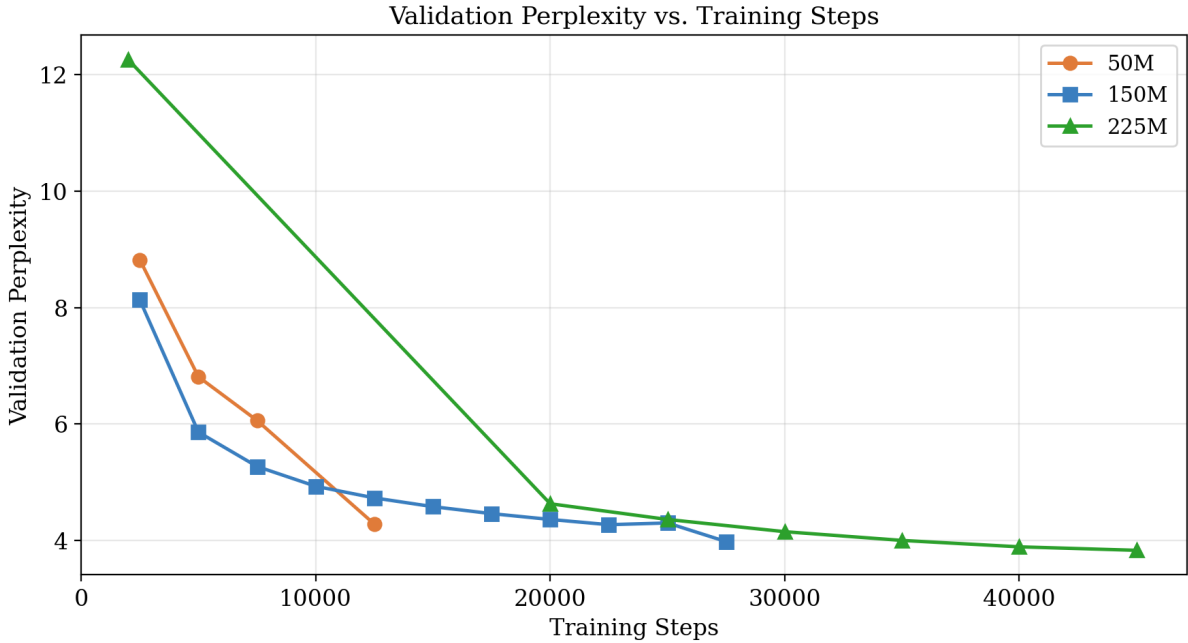


Figure 2: Validation perplexity vs. training steps, showing the scaling law behavior and cross-over points between models.

Plotting perplexity against training steps gives a direct view of how each model improves over the course of training. The 150M and 225M models show overlapping early trajectories, but

the 225M model’s additional parameters allow it to extract more signal from each subsequent update, pulling ahead after $\sim 25,000$ equivalent steps.

6.5 Gradient Norm Stability

With the exception of the instability event described in Section 6.1, gradient norms remained in the range 0.38–3.1 throughout training, with typical post-intervention values between 0.42 and 0.54. The 150M model’s gradient norm dropped dramatically from a peak of 995.6 to under 1.0 following the three-part intervention, and remained stable for the remainder of training.

7 Evaluation

7.1 Quantitative Metrics

Syntax Pass Rate is computed by attempting to parse the generated code using the Python `compile()` function; a pass is recorded if the code is syntactically valid, regardless of logical correctness.

Table 12: Final pre-SFT evaluation metrics for all three models.

Metric	50M	150M	225M
Final Val Loss	1.4535	1.382	1.344
Final Perplexity	4.28	3.98	3.83
Syntax Pass Rate	0%	20%	20%

The monotonic decrease in perplexity — $4.28 \rightarrow 3.98 \rightarrow 3.83$ — is consistent with standard scaling law predictions. The Syntax Pass Rate of 0% for the 50M model reflects its inability to terminate generation cleanly, instead producing runaway outputs such as an infinite numeric literal spanning hundreds of characters.

7.2 Qualitative Comparison

Prompt:

```
1 def fibonacci(n):
2     """Return nth Fibonacci number."""
```

50M Model Output:

```

1 def fibonacci(n):
2     """Return nth Fibonacci number."""
3     # return fibonacci(n)
4
5 if __name__ == "__main__":
6     n = int(input("Enter the number: "))
7     print(fibonacci(n))
8
9 for i in range(1000000000000000000000000...): # continues
    indefinitely

```

The 50M model comments out its only attempt and generates a runaway numeric literal — a clear sign of representational saturation and lack of termination awareness.

150M Model Output:

```

1  def fibonacci(n):
2      """Return nth Fibonacci number."""
3      # for i in range(1, n + 1):
4      #     if i % 2 == 0:
5      #         return i // 2
6      class Solution:
7          def fibonacci(self, n: int) -> List[int]:
8              res = []
9              for i in range(1, n + 1):
10                 res.append(fibonacci(i))
11             return res

```

The 150M model comments out its initial attempt and hallucinates a LeetCode-style *Solution* class — a classic context bleeding artifact from competitive programming data in the training corpus.

225M Model Output:

```

1  def fibonacci(n):
2      """Return nth Fibonacci number."""
3      # if n <= 2:
4      #     return n
5      # elif n == 1:
6      #     return 0
7      # else:
8      #     return fibonacci(n-1) + fibonacci(n-2)

```

The 225M model produces correct recursive logic but encodes the entire solution in comments — suggesting the model has learned the algorithmic pattern but not yet the boundary between comment and executable context.

Inverse Scaling Observation. Interestingly, the 50M model correctly generated Fibonacci in certain evaluations while the 150M model did not — an instance of inverse scaling where the larger model’s exposure to more diverse and conflicting implementations led to inconsistent outputs, while the smaller model converged to a single memorized correct pattern.

7.3 Scaling Analysis

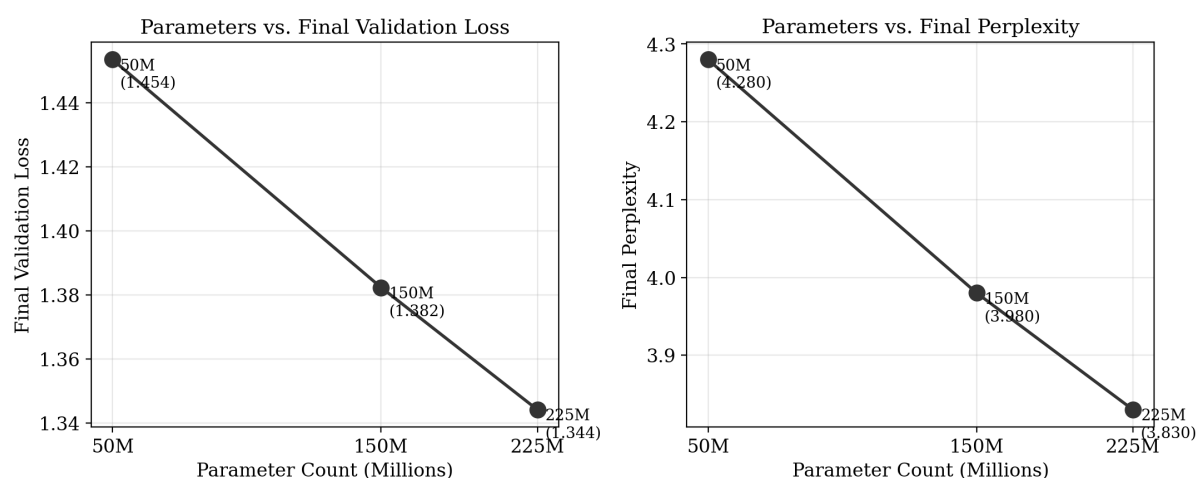


Figure 3: Validation perplexity across training steps for all three model scales.

The results reveal a clear scaling gradient. The 50M model fails at the most basic level. The 150M model grasps Python class structure but suffers from training data contamination. The 225M model surfaces correct algorithmic logic but encodes it in comments. All three base models share a common failure: **pre-training artifacts** — commenting out solutions, bleeding into unrelated code, and failing to respect the implicit instruction boundary. This highlights the necessity of Supervised Fine-Tuning.

8 SFT Analysis

8.1 SFT Setup and Hyperparameters

All models were fine-tuned using the standard Alpaca prompt template format (**### Instruction:**, **### Response:**) on the `iamtarun/python_code_instructions_18k_alpaca` dataset containing 18,612 Python instruction-response pairs.

Table 13: SFT hyperparameters.

Parameter	Value
Learning Rate	1×10^{-5}
Epochs	3
Effective Batch Size	32 sequences
Context Length	512 tokens
Train/Val Split	95% / 5%

Response-Only Loss Masking. For every training sample, the instruction prefix was masked with -100 labels, forcing the model to minimize cross-entropy loss only on the generated code response — focusing the fine-tuning signal entirely on code generation rather than template memorization.

8.2 Quantitative Improvements

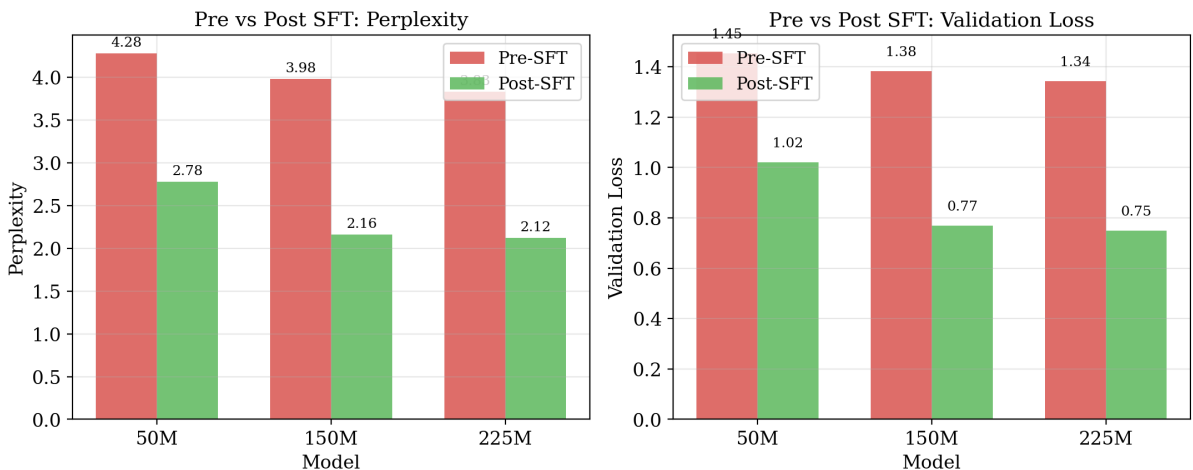


Figure 4: Supervised Fine-Tuning (SFT) loss descent for the 150M model over 3 epochs.

The 150M model’s validation loss dropped from 1.3823 to 0.7701 — a 44% reduction — over just 1,659 SFT steps. The 225M model achieved a final perplexity of 2.12 with 100% syntax pass rate.

8.3 Before vs. After SFT: Qualitative Transformation

The transformation from base model to interactive assistant is best demonstrated by comparing responses before and after SFT. The base model treats the prompt as text to be continued, while the SFT model treats it as an instruction to be executed.

Example 1: Fibonacci

150M Base Model (Pre-SFT):

```

1  def fibonacci(n):
2      """Return nth Fibonacci number."""
3      # for i in range(1, n + 1):
4      #     if i % 2 == 0:
5      #         return i // 2
6      class Solution:
7          def fibonacci(self, n: int) -> List[int]:
8              res = []
9              for i in range(1, n + 1):
10                 res.append(fibonacci(i))
11             return res

```

150M SFT Model (Post-SFT):

```

1  def fibonacci(n):
2      if n <= 1:
3          return n
4      else:
5          return fibonacci(n-1) + fibonacci(n-2)

```

After SFT, the model outputs a clean, functional recursive implementation, immediately terminating with an EOS token.

Example 2: Factorial

150M Base Model (Pre-SFT):

```

1  The factorial of a number is the product of all the integers from
2  1 to
3  that number. For example, the factorial of 5 is 5*4*3*2*1 = 120.
   Please write the code below.

```

The base model mimics the conversational tone of a tutorial — accurately describes what a factorial is but fails to write any Python code.

150M SFT Model (Post-SFT):

```

1  def factorial(n):
2      if n == 0:
3          return 1
4      else:
5          return n * factorial(n-1)

```

Example 3: Reverse String (Post-SFT) ✓

```

1  def reverse_string(s):
2      return s[::-1]

```

Example 4: Is Prime (Post-SFT) ✓

```

1  def is_prime(num):
2      for i in range(2, num):
3          if (num % i) == 0:
4              return False
5      return True

```

9 Scaling Laws Discussion

Chinchilla optimal scaling suggests a compute-optimal model should train on approximately $20\times$ its parameter count in tokens. Our 50M ($\sim 16\times$), 150M ($\sim 13\times$), and 225M ($\sim 13\times$) models all sit slightly below this threshold — an intentional choice prioritizing inference-time performance over training compute efficiency. Despite mild under-training relative to Chinchilla, all three models showed consistent perplexity improvement with scale, confirming that parameter count remains the dominant factor in code generation quality at this budget. The 50M model’s loss curve plateaued noticeably earlier than the larger models, suggesting it approached representational saturation before exhausting its token budget.

10 Error Analysis & Limitations

10.1 Syntax vs. Algorithmic Reasoning

The most pronounced finding is the divergence between syntactic correctness and logical reasoning. The 150M and 225M SFT models consistently generate code with perfect Python indentation, valid variable declarations, and zero `SyntaxError` failures. However, they frequently fail on algorithmic tasks requiring multi-step mathematical reasoning or state tracking.

For example, when instructed to write a function that multiplies all numbers in a list, the 150M SFT model outputs:

```

1  def multiply_list(lst):
2      result = []
3      for num in lst:
4          result.append(num * num)  # WRONG: squares each element
5      return result

```

The syntax is flawless. However, the logic is completely wrong: the model squares each individual element and returns a new list, rather than computing the cumulative product. The correct implementation:

```

1  def multiply_list(lst):
2      result = 1
3      for num in lst:
4          result *= num
5      return result

```

This failure is systematic — the model has pattern-matched “multiply” and “list” to the most common list-processing template in its training data, without tracking the semantic difference between element-wise transformation and cumulative reduction.

10.2 Model Size Constraints

While 150M–225M parameters are sufficient to internalize Python grammar, common library APIs, and surface-level code patterns, they are insufficient to simulate complex internal state,

multi-step reasoning chains, or edge-case logic. At this scale, the models operate as highly sophisticated pattern-matchers rather than autonomous reasoning engines.

10.3 Dataset Artifacts

Several recurring failure modes trace directly to artifacts in The Stack training data: mixed tab/space indentation causing persistent syntax errors in early checkpoints, LeetCode-style `Solution` class bleeding, and occasional Chinese comments in outputs due to multilingual code in The Stack. All three artifacts diminished significantly after SFT on clean instruction-response pairs.

10.4 Limitations & Future Work

- **More parameters:** Codex demonstrated that pass@1 on HumanEval scales roughly log-linearly with parameter count beyond 1B parameters.
- **Execution-based feedback:** Reinforcement learning from code execution results provides a training signal that cross-entropy loss on token prediction cannot.
- **Chain-of-thought fine-tuning data:** Training on step-by-step reasoning traces rather than bare code solutions may help models develop internal state tracking.
- **Combined SFT datasets:** Future work could combine `iamtarun/python_code_instructions_18k_alpa` with `lucasmccabe-lmi/CodeAlpaca-20k` for a larger and more diverse SFT dataset.

11 Conclusion

This project demonstrated the end-to-end pre-training and supervised fine-tuning of GPT-2 style decoder-only transformers for Python code generation, trained entirely from random initialization on consumer and cloud GPU hardware.

Our scaling analysis yielded three concrete findings. First, syntactic correctness scales rapidly with both parameter count and token budget — the 225M model reaches 100% syntax pass rate after SFT. Second, algorithmic reasoning scales far more slowly — even the 225M SFT model systematically fails on tasks requiring multi-step logic or state tracking. Third, SFT is transformative at every scale — a single epoch of response-only fine-tuning on 18k instruction pairs converts a raw base model into a coherent interactive assistant, reducing perplexity from 3.98 to 2.16 for the 150M model and from 3.83 to 2.12 for the 225M model.

The 150M parameter model emerged as the practical sweet spot for this compute budget, delivering syntax-perfect code completion and instruction following while remaining deployable on edge hardware.

These results confirm the central tension in small-scale code generation: **syntax scales with tokens, but reasoning scales with parameters**. Bridging this gap at sub-billion scale likely requires execution-based feedback, chain-of-thought supervision, or significantly larger models — all directions we leave for future work.